

Yep. For **Experiment 9 — Discrete Hopfield Network: Pattern Storage and Recall**, here's the same proper lab-record format as Experiment 8, based directly on your handout.

EXPERIMENT 9

Discrete Hopfield Network – Pattern Storage and Recall

1. Aim

To implement a **Discrete Hopfield Network** in MATLAB for storing the binary pattern (**1 1 1 0**) and retrieving the original pattern when the test input contains mistakes in the first and second positions.

3. Software Requirements

- MATLAB
 - Computer system with Windows/Linux/macOS
-

4. Theory

A **Hopfield Network** is a recurrent neural network primarily used as an **associative memory**. It stores patterns in the connections between its neurons and can retrieve a stored pattern when a noisy or partially incorrect version of that pattern is provided as input.

A **Discrete Hopfield Network** consists of interconnected neurons whose states are generally represented using **bipolar values**, **+1** and **-1**. Therefore, binary **1** is represented by **+1**, while binary **0** is represented by **-1**.

In this experiment, the binary pattern:

[1 1 1 0]

is converted into the bipolar representation:

[1 1 1 -1]

The network stores the pattern by constructing a **weight matrix using the Hebbian learning rule**. For a single bipolar pattern XX , the weight matrix is calculated as:

$$W = XX^T - I$$

The diagonal elements of the weight matrix are then set to zero because a Hopfield neuron does not have a connection with itself.

During recall, a noisy input is given to the network. The network calculates the weighted input received by each neuron and applies a bipolar activation function. The neurons update their states based on the sign of their net input.

The network therefore acts as an **associative memory**, allowing it to reconstruct the original stored pattern from an incorrect or noisy input.

5. Given Pattern

Stored Binary Pattern

[1 1 1 0]

Bipolar Representation

[1 1 1 -1]

In bipolar representation:

1 $\rightarrow$ +1

0 $\rightarrow$ -1

6. Weight Matrix Calculation

The Hebbian learning rule for one stored bipolar pattern is:

$$W = XT X^T$$

For the pattern:

$$X = [1 \ 1 \ 1 \ -1]$$

After calculating XTX^T , the diagonal elements are set to zero to remove self-connections.

The resulting weight matrix is:

$$W = \begin{bmatrix} 0 & 1 & 1 & -1 \\ 1 & 0 & 1 & -1 \\ 1 & 1 & 0 & -1 \\ -1 & -1 & -1 & 0 \end{bmatrix}$$

9. Algorithm

1. Start the MATLAB program.
2. Define the stored binary pattern $[1 \ 1 \ 1 \ 0]$.
3. Convert the binary pattern into bipolar form.
4. Calculate the weight matrix using the Hebbian learning rule.
5. Set the diagonal elements of the weight matrix to zero.
6. Define the noisy input $[0 \ 0 \ 1 \ 0]$.
7. Convert the noisy input into bipolar form.
8. Calculate the net input using $\text{Net} = W \times X_{\text{Net}} = W \times X$.
9. Apply the bipolar $\text{sign}()$ activation function.
10. Convert the bipolar output back into binary form.
11. Compare the recovered pattern with the original stored pattern.
12. Display the result.
13. Stop.

13. Conclusion

This experiment demonstrates the **associative memory property of a Discrete Hopfield Network**. The network uses a weight matrix learned using the Hebbian rule to reconstruct a stored pattern from a noisy input.